

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: Fundamentals of Data Science

COURSE CODE: DSA04

COURSE OUTCOME COVERED

CO4: Develop machine learning models for classification, regression, and clustering, including performance evaluation and methodology comparison. (BL3)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:100

**Annexure A
Industry Problem Solving**

Code of Conduct:

I VIVETHA B 192424170 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VIVETHA B

RUBRICS FOR CALCULATION

SCENARIO BASED ASSESSMENT							
Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Level	Marks
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario		
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues		
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts		
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided		
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand		

Annexure A

Question 1: Real Estate Price Prediction Using Linear Regression

A real estate company wants to predict the **selling price of houses** based on house-related features.

House	Area sq.ft	Bedrooms	Age	Distance from City km	Price Lakhs
H1	900	2	12	10	45
H2	1100	2	10	8	55
H3	1300	3	8	6	68
H4	1500	3	5	5	82
H5	1800	4	3	3	115
H6	2000	4	2	2	135
H7	1000	2	15	12	40
H8	1600	3	6	4	90

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {
```

```
    "House": ["H1", "H2", "H3", "H4", "H5", "H6", "H7", "H8"],
```

```
    "Area": [900, 1100, 1300, 1500, 1800, 2000, 1000, 1600],
```

```
    "Bedrooms": [2, 2, 3, 3, 4, 4, 2, 3],
```

```
    "Age": [12, 10, 8, 5, 3, 2, 15, 6],
```

```
    "Distance": [10, 8, 6, 5, 3, 2, 12, 4],
```

```
    "Price": [45, 55, 68, 82, 115, 135, 40, 90]}
```

```
df = pd.DataFrame(data)
```

```
print("Real Estate Dataset:")
```

```
print(df)
```

Real Estate Dataset:

House Area Bedrooms Age Distance Price

S.NO	HOUSE	AREA	BEDROOMS	AGE	DISTANCE	PRICE
0	H1	900	2	12	10	45
1	H2	1100	2	10	8	44
2	H3	1300	3	8	6	68
3	H4	1500	3	5	5	82
4	H5	1800	4	3	3	115
5	H6	2000	4	2	2	135
6	H7	1000	2	15	12	40
7	H8	1600	3	6	4	90

b) Identify independent and dependent variables.

Independent Variables (Input Features — X)

These are the variables used to **predict the house price**:

1. **Area** – House area in square feet
2. **Bedrooms** – Number of bedrooms
3. **Age** – Age of the house in years
4. **Distance** – Distance from the city in km

Therefore:

$X = [\text{Area, Bedrooms, Age, Distance}]$

Dependent Variable (Output — Y)

The variable that we want to **predict** is:

- **Price** – Selling price of the house in lakhs

Therefore:

$Y = \text{Price}$

c) Split the dataset into training and testing data.

The dataset is divided into **training data** and **testing data**.

- **Training data:** Used to train the Linear Regression model.
- **Testing data:** Used to evaluate how well the trained model predicts new/unseen data.
- Here, we use **75% for training** and **25% for testing**.

Python Code

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X, y,

    test_size=0.25,

    random_state=42)

print("Training Data:")

print(X_train)

print("\nTesting Data:")

print(X_test)

print("\nTraining Target Values:")

print(y_train)

print("\nTesting Target Values:")

print(y_test)
```

Explanation

test_size=0.25 means **25% of the dataset is used for testing** and the remaining **75% is used for training**.

Since the dataset has **8 houses**:

- **Training data = 6 houses**
- **Testing data = 2 houses**

random_state=42 ensures that the same houses are selected for training and testing every time you run the code.

d) Implement **Linear Regression**.

In this step, we create a **Linear Regression model** and train it using the training data obtained in Question C.

Python Code

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()

model.fit(X_train, y_train)

print("Linear Regression model trained successfully!")
```

To Display the Model Coefficients

```
print("Coefficients:")

print(model.coef_)

print("\nIntercept:")

print(model.intercept_)
```

Explanation

The Linear Regression model learns the relationship between the independent variables:

- **Area**
- **Bedrooms**
- **Age**
- **Distance**

and the dependent variable:

- **Price**

The general Linear Regression equation is:

$$\text{Price} = b_0 + b_1(\text{Area}) + b_2(\text{Bedrooms}) + b_3(\text{Age}) + b_4(\text{Distance})$$

Where:

- b_0 = Intercept
- b_1, b_2, b_3, b_4 = Coefficients learned by the model

After training, the model can be used to **predict house prices** for the test data and new houses.

e) Predict house price for test data.

After training the Linear Regression model, we use the **testing data (X_test)** to predict the house prices.

Python Code

```
y_pred = model.predict(X_test)

print("Actual vs Predicted House Prices:")

for actual, predicted in zip(y_test, y_pred):

    print("Actual Price:", actual, "Lakhs")

    print("Predicted Price:", round(predicted, 2), "Lakhs")

    print()
```

Output

Actual vs Predicted House Prices:

Actual Price: 55 Lakhs

Predicted Price: 51.19 Lakhs

Actual Price: 135 Lakhs

Predicted Price: 121.84 Lakhs

In Table Form

House	Actual Price (Lakhs)	Predicted Price (Lakhs)
-------	----------------------	-------------------------

H2	55	51.19
----	----	-------

H6	135	121.84
----	-----	--------

Explanation

The trained Linear Regression model predicts the prices of the **two houses in the testing dataset**.

- For **H2**, the actual price is ₹55 lakhs and the predicted price is approximately **₹51.19 lakhs**.
- For **H6**, the actual price is ₹135 lakhs and the predicted price is approximately **₹121.84 lakhs**.

The predicted values are reasonably close to the actual values, indicating that the model has learned a relationship between the house features and selling price.

f) Predict the price of a new house with:

Area	Bedrooms	Age	Distance
1700	3	4	4

We need to predict the selling price of a **new house** with the following details:

Area (sq.ft) Bedrooms Age (years) Distance (km)

1700 3 4 4

Python Code

```
new_house = pd.DataFrame({
    "Area": [1700],
    "Bedrooms": [3],
    "Age": [4],
    "Distance": [4]
})

new_price = model.predict(new_house)

print("New House Details:")

print(new_house)

print("\nPredicted Price:")

print(round(new_price[0], 2), "Lakhs")
```

Output

New House Details:

```
Area Bedrooms Age Distance
0 1700      3    4      4
```

Predicted Price:

95.95 Lakhs

Explanation

The trained Linear Regression model uses the new house's:

- **Area = 1700 sq.ft**
- **Bedrooms = 3**
- **Age = 4 years**
- **Distance = 4 km**

to predict its selling price.

Therefore, the **predicted price of the new house is approximately ₹95.95 lakhs.**

g) Evaluate the model using **MAE, MSE, RMSE, and R² score.**

The Linear Regression model is evaluated using **MAE, MSE, RMSE, and R² Score.**

Evaluation Metric	Value
MAE	8.49 Lakhs
MSE	93.92
RMSE	9.69 Lakhs
R ² Score	0.9413

Interpretation

- **MAE = 8.49 Lakhs:** On average, the predicted house price differs from the actual price by about **₹8.49 lakhs.**
- **MSE = 93.92:** It represents the average squared prediction error. Lower MSE indicates better performance.
- **RMSE = 9.69 Lakhs:** The typical prediction error is approximately **₹9.69 lakhs.**
- **R² Score = 0.9413:** The model explains approximately **94.13% of the variation in house prices.**

Therefore, the model shows **good performance on the given dataset.**

h) Interpret the result and explain whether the model is useful for real estate price prediction.

The Linear Regression model performs well for the given real estate dataset. The **R² score of 0.9413** indicates that approximately **94.13% of the variation in house prices** can be explained by the given features: area, number of bedrooms, age, and distance from the city.

The **MAE of 8.49 lakhs** and **RMSE of 9.69 lakhs** indicate that the model's predictions are reasonably close to the actual house prices.

For the new house with **1700 sq.ft area, 3 bedrooms, 4 years of age, and 4 km distance from the city**, the predicted selling price is approximately **₹95.95 lakhs**.

Final Conclusion

The model is **useful for basic real estate price prediction** because it provides reasonably accurate predictions and has a high R^2 score. However, the dataset contains only **8 houses**, which is very small. Therefore, for real-world use, a much larger dataset with additional features such as location, bathrooms, parking, property type, and amenities would be required.

Question 2: Customer Churn Prediction Using Logistic Regression

A telecom company wants to predict whether a customer will **churn** or **not churn**.

Customer	Monthly Bill	Complaints	Data Usage GB	Tenure Months	Churn
C1	500	0	25	36	0
C2	900	3	10	8	1
C3	750	2	15	12	1
C4	400	0	30	48	0
C5	1000	4	8	6	1
C6	550	1	22	30	0
C7	850	3	12	10	1
C8	450	0	28	40	0

Here,

Churn = 1 means customer leaves the company.

Churn = 0 means customer stays.

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {
```

```
    "Customer": ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8"],
```

```
    "Monthly_Bill": [500, 900, 750, 400, 1000, 550, 850, 450],
```

```
    "Complaints": [0, 3, 2, 0, 4, 1, 3, 0],
```

```
    "Data_Usage_GB": [25, 10, 15, 30, 8, 22, 12, 28],
```

```
    "Tenure_Months": [36, 8, 12, 48, 6, 30, 10, 40],
```

```
"Churn": [0, 1, 1, 0, 1, 0, 1, 0]
}

df = pd.DataFrame(data)

print("Customer Churn Dataset:")

print(df)
```

b) Identify the input features and target variable.

Input Features (Independent Variables)

The input features used to predict customer churn are:

- **Monthly Bill**
- **Complaints**
- **Data Usage (GB)**
- **Tenure (Months)**

Therefore:

X=[Monthly Bill, Complaints, Data Usage, Tenure]

Target Variable (Dependent Variable)

The target variable is:

- **Churn**

Where:

- **Churn = 1** → Customer leaves
- **Churn = 0** → Customer stays

Therefore:

Y=Churn

Python Code

```
X = df[["Monthly_Bill", "Complaints", "Data_Usage_GB", "Tenure_Months"]]

y = df["Churn"]

print("Input Features:")

print(X)
```

```
print("\nTarget Variable:")
```

```
print(y)
```

c) Perform train-test split.

The dataset is divided into:

- **75% training data**
- **25% testing data**

Since there are 8 customers:

- Training = **6 customers**
- Testing = **2 customers**

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y,
```

```
    test_size=0.25,
```

```
    random_state=42
```

```
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

d) Implement **Logistic Regression**.

Logistic Regression is used because the target variable has two possible outcomes: **0 or 1**.

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Logistic Regression model trained successfully.")
```

e) Predict churn status for test customers.

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Churn:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual:", actual, "Predicted:", predicted)
```

Output

Customer	Actual Churn	Predicted Churn
----------	--------------	-----------------

C2	1	1
----	---	---

C6	0	0
----	---	---

So:

- **C2 → Churn**

C6 → Not Churn

f) Predict churn for a new customer:

Monthly Bill	Complaints	Data Usage GB	Tenure Months
800	2	14	10

Python Code

```
new_customer = pd.DataFrame({
```

```
    "Monthly_Bill": [800],
```

```
    "Complaints": [2],
```

```
    "Data_Usage_GB": [14],
```

```
    "Tenure_Months": [10]
```

```
})
```

```
new_prediction = model.predict(new_customer)
```

```
print("New Customer:")
```

```
print(new_customer)

print("\nPredicted Churn Status:")

if new_prediction[0] == 1:

    print("Customer will CHURN")

else:

    print("Customer will NOT CHURN")
```

Output

Predicted Churn Status:

Customer will CHURN

Therefore, the new customer is predicted to **churn (1)**.

g) Evaluate the model using **accuracy, confusion matrix, precision, recall, and F1-score**.

The Logistic Regression model is evaluated using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

Python Code

```
from sklearn.metrics import (

    accuracy_score,

    confusion_matrix,

    precision_score,

    recall_score,

    f1_score

)

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

precision = precision_score(y_test, y_pred)
```

```

recall = recall_score(y_test, y_pred)

f1 = f1_score(y_test, y_pred)

print("Model Evaluation:")

print("Accuracy :", accuracy)

print("\nConfusion Matrix:")

print(cm)

print("\nPrecision:", precision)

print("Recall  :", recall)

print("F1 Score :", f1)

```

Output

Accuracy : 1.0

Confusion Matrix:

```
[[1 0]
```

```
[0 1]]
```

Precision: 1.0

Recall : 1.0

F1 Score : 1.0

Interpretation

Metric	Value	Interpretation
Accuracy	1.00 (100%)	All test customers were correctly classified
Precision	1.00 (100%)	All customers predicted as churn actually churned
Recall	1.00 (100%)	All actual churn customers were correctly identified

F1-score **1.00 (100%)** Excellent balance between precision and recall

Confusion Matrix

	Predicted		
	0	1	
Actual	0	1	0
	1	0	1

There are:

- **1 True Negative**
- **1 True Positive**
- **0 False Positive**

0 False Negative

h) Explain why Logistic Regression is suitable for churn prediction.

Logistic Regression is suitable for customer churn prediction because **churn is a binary classification problem**.

The target variable has only two possible outcomes:

- **0 → Customer stays**
- **1 → Customer churns**

Logistic Regression calculates the **probability of a customer churning** and classifies the customer as either churn or not churn based on that probability.

It can use factors such as **monthly bill, number of complaints, data usage, and customer tenure** to identify customers who are more likely to leave.

Conclusion

Logistic Regression is a simple and effective method for predicting customer churn. In this dataset, the model achieved **100% accuracy, precision, recall, and F1-score** on the test data. However, the dataset contains only **8 customers**, so these perfect scores should not be considered representative of real-world performance. A larger dataset would be required for reliable telecom churn prediction.

Question 3: Marketing Sales Prediction Using Lasso Regression

A retail company wants to predict **monthly sales** based on advertisement spending across different platforms. The company wants to know which marketing channels are most important.

Month	TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns	Sales Lakhs
M1	50	20	10	5	15
M2	60	25	12	6	18
M3	80	40	15	8	25
M4	100	60	20	10	32
M5	120	70	22	12	38
M6	40	15	8	4	12
M7	90	50	18	9	28
M8	110	65	21	11	35

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {
    "Month": ["M1", "M2", "M3", "M4", "M5", "M6", "M7", "M8"],
    "TV_Ads": [50, 60, 80, 100, 120, 40, 90, 110],
    "Social_Media_Ads": [20, 25, 40, 60, 70, 15, 50, 65],
    "Newspaper_Ads": [10, 12, 15, 20, 22, 8, 18, 21],
    "Email_Campaigns": [5, 6, 8, 10, 12, 4, 9, 11],
    "Sales": [15, 18, 25, 32, 38, 12, 28, 35]
}
```

```
df = pd.DataFrame(data)
```

```
print("Marketing Sales Dataset:")
```

```
print(df)
```

b) Identify features and target variable.

Features / Independent Variables

The features used to predict monthly sales are:

1. **TV Ads**
2. **Social Media Ads**
3. **Newspaper Ads**
4. **Email Campaigns**

Therefore:

X=[TV, Social Media, Newspaper, Email]

Target Variable

The target variable is:

- **Sales (Lakhs)**

Therefore:

Y=Sales

Python Code

```
X = df[["TV_Ads", "Social_Media_Ads", "Newspaper_Ads", "Email_Campaigns"]]
```

```
y = df["Sales"]
```

```
print("Features:")
```

```
print(X)
```

```
print("\nTarget Variable:")
```

```
print(y)
```

c) Apply train-test split.

The dataset is divided into **75% training data** and **25% testing data**.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.25,
```

```
    random_state=42
```

```
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

There are 8 records:

- **Training data = 6 records**
- **Testing data = 2 records**

d) Implement **Lasso Regression**.

Lasso Regression is a regularized form of Linear Regression that can reduce the importance of less useful features.

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=0.1)
```

```
model.fit(X_train, y_train)
```

```
print("Lasso Regression model trained successfully.")
```

e) Use an alpha value such as **0.1 or 1.0**.

Lasso Regression is a regularized form of Linear Regression that can reduce the importance of less useful features.

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=0.1)
```

```
model.fit(X_train, y_train)
```

```
print("Lasso Regression model trained successfully.")
```

f) Predict sales for test data.

```
# Predict sales for test data
```

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Sales:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual Sales:", actual, "Lakhs")
```

```
    print("Predicted Sales:", round(predicted, 2), "Lakhs")
```

```
    print()
```

Output

Month	Actual Sales	Predicted Sales
-------	--------------	-----------------

M2	18	18.18
----	----	-------

M6	12	11.91
----	----	-------

The predicted values are very close to the actual sales values.

g) Predict sales for a new campaign:

TV Ads	Social Media Ads	Newspaper Ads	Email Campaigns
95	55	18	9

```
new_campaign = pd.DataFrame({
```

```
    "TV_Ads": [95],
```

```
    "Social_Media_Ads": [55],
```

```
    "Newspaper_Ads": [18],
```

```
    "Email_Campaigns": [9]
```

```
})
```

```
new_sales = model.predict(new_campaign)

print("New Campaign:")

print(new_campaign)

print("\nPredicted Sales:")

print(round(new_sales[0], 2), "Lakhs")
```

Output

Predicted Sales:

30.06 Lakhs

Therefore, the predicted monthly sales for the new campaign are approximately **₹30.06 lakhs**.

h) Evaluate using **MAE, MSE, RMSE, and R² score**.

The model is evaluated using:

- MAE
- MSE
- RMSE
- R² Score

Python Code

```
from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Model Evaluation:")

print("MAE :", round(mae, 4))

print("MSE :", round(mse, 4))
```

```
print("RMSE :", round(rmse, 4))
```

```
print("R2 Score:", round(r2, 4))
```

Output

Metric	Value
MAE	0.1338 Lakhs
MSE	0.0198
RMSE	0.1409 Lakhs
R ² Score	0.9978

Interpretation

- **MAE = 0.1338 lakhs:** The average prediction error is approximately ₹0.134 lakhs.
- **MSE = 0.0198:** The squared prediction error is very small.
- **RMSE = 0.1409 lakhs:** The typical prediction error is approximately ₹0.141 lakhs.
- **R² = 0.9978:** The model explains approximately **99.78% of the variation in sales** on the test data

i) Display the coefficients and identify which features are reduced or selected by Lasso.

```
print("Lasso Regression Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_, 4))
```

Output

Lasso Regression Coefficients:

TV_Ads : 0.2768

Social_Media_Ads : 0.0732

Newspaper_Ads : 0.0000

Email_Campaigns : 0.0000

Intercept: -0.2589

Interpretation

Feature	Coefficient	Status
TV Ads	0.2768	Selected
Social Media Ads	0.0732	Selected
Newspaper Ads	0.0000	Reduced/Removed
Email Campaigns	0.0000	Reduced/Removed

Lasso has reduced the coefficients of **Newspaper Ads** and **Email Campaigns** to zero.

Therefore, for this particular dataset and $\alpha = 0.1$, Lasso identifies:

- **TV Ads** → Important feature
- **Social Media Ads** → Important feature
- **Newspaper Ads** → Reduced/removed
- **Email Campaigns** → Reduced/removed

j) Explain how Lasso helps in feature selection.

Lasso Regression adds a **penalty** to the model that discourages large coefficients.

The Lasso objective function is:

Minimize Error + $\alpha \sum |\text{Coefficient}|$

Because of this penalty, some coefficients can become exactly **zero**.

When a coefficient becomes zero, that feature is effectively removed from the model.

In this example:

TV Ads and **Social Media Ads** have non-zero coefficients, so they are selected as useful predictors.

Newspaper Ads and **Email Campaigns** have coefficients of zero, so Lasso has reduced their importance and removed them from the model.

Conclusion

Lasso Regression is useful because it performs **prediction and feature selection at the same time**. It helps the company identify which advertising channels contribute most to sales while reducing the influence of less important features. In this dataset, **TV advertising and Social Media advertising are the most important selected features**.

Question 4: Electricity Consumption Prediction Using Ridge Regression

A smart energy company wants to predict household **electricity consumption**. Some features may be highly correlated, such as number of appliances and daily usage hours. Therefore, Ridge Regression is used.

House	Appliances	Usage Hours	House Size sq.ft	Occupants	Consumption Units
H1	5	4	800	3	120
H2	7	6	1000	4	190
H3	8	7	1200	4	240
H4	10	8	1500	5	320
H5	4	3	700	2	90
H6	6	5	900	3	160
H7	9	7	1300	5	280
H8	11	9	1600	6	350

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {  
    "House": ["H1", "H2", "H3", "H4", "H5", "H6", "H7", "H8"],  
    "Appliances": [5, 7, 8, 10, 4, 6, 9, 11],  
    "Usage_Hours": [4, 6, 7, 8, 3, 5, 7, 9],  
    "House_Size": [800, 1000, 1200, 1500, 700, 900, 1300, 1600],  
    "Occupants": [3, 4, 4, 5, 2, 3, 5, 6],  
    "Consumption": [120, 190, 240, 320, 90, 160, 280, 350]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Electricity Consumption Dataset:")
```

```
print(df)
```

b) Identify the independent and dependent variables.

Independent Variables

The features used to predict electricity consumption are:

1. **Appliances**
2. **Usage Hours**
3. **House Size**
4. **Occupants**

Therefore:

```
X=[Appliances, Usage Hours, House Size, Occupants]
```

Dependent Variable

The variable we want to predict is:

- **Consumption (Units)**

Therefore:

```
Y=Consumption
```

Python Code

```
X = df[["Appliances", "Usage_Hours", "House_Size", "Occupants"]]
```

```
y = df["Consumption"]
```

```
print("Independent Variables:")
```

```
print(X)
```

```
print("\nDependent Variable:")
```

```
print(y)
```

c) Split the data into training and testing sets.

We use **75% of the data for training** and **25% for testing**.

```
from sklearn.model_selection import train_test_split
```



```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42  
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

Since there are 8 houses:

- **Training data = 6 houses**
- **Testing data = 2 houses**

d) Implement **Ridge Regression**.

Ridge Regression is a regularized version of Linear Regression. It is useful when some independent variables are highly correlated.

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
print("Ridge Regression model trained successfully.")
```

e) Use an alpha value such as **1.0**.

Here we use:

$\alpha=1.0$

alpha = 1.0

```
model = Ridge(alpha=alpha)
```

```
model.fit(X_train, y_train)
```

```
print("Alpha value:", alpha)
```

A larger alpha value applies stronger regularization to the model.

f) Predict electricity consumption for test data.

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Consumption:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual Consumption:", actual, "Units")
```

```
    print("Predicted Consumption:", round(predicted, 2), "Units")
```

```
    print()
```

Output

House	Actual Consumption	Predicted Consumption
-------	--------------------	-----------------------

H2	190	184.62
----	-----	--------

H6	160	152.45
----	-----	--------

The predictions are reasonably close to the actual consumption values.

g) Predict consumption for a new house:

Appliances	Usage Hours	House Size	Occupants
8	6	1100	4

```
new_house = pd.DataFrame({
```

```
    "Appliances": [8],
```

```

    "Usage_Hours": [6],
    "House_Size": [1100],
    "Occupants": [4]
})

new_consumption = model.predict(new_house)

print("New House Details:")

print(new_house)

print("\nPredicted Electricity Consumption:")

print(round(new_consumption[0], 2), "Units")

```

OUTPUT

Predicted Electricity Consumption:

212.73 Units

Therefore, the predicted electricity consumption for the new house is approximately **212.73 units**. The model is evaluated using:

h) Evaluate using **MAE, MSE, RMSE, and R² score**.

- **MAE**
- **MSE**
- **RMSE**
- **R² Score**

Python Code

```

from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

```

```
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
```

```
r2 = r2_score(y_test, y_pred)
```

```
print("Model Evaluation:")
```

```
print("MAE :", round(mae, 2))
```

```
print("MSE :", round(mse, 2))
```

```
print("RMSE :", round(rmse, 2))
```

```
print("R2 Score:", round(r2, 4))
```

Output

Metric	Value
--------	-------

MAE	6.46 Units
-----	------------

MSE	42.94
-----	-------

RMSE	6.55 Units
------	------------

R ² Score	0.8092
----------------------	--------

Interpretation

- **MAE = 6.46:** On average, the predicted electricity consumption differs from the actual value by approximately **6.46 units**.
- **MSE = 42.94:** This represents the average squared prediction error.
- **RMSE = 6.55:** The typical prediction error is approximately **6.55 units**.
- **R² = 0.8092:** The model explains approximately **80.92% of the variation in electricity consumption**.

i) Display the Ridge coefficients.

Python Code

```
print("Ridge Regression Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_, 4))
```

Output

Ridge Regression Coefficients:

Appliances : 3.6817

Usage_Hours : 1.0696

House_Size : 0.2443

Occupants : 2.9836

Intercept: -103.8165

Interpretation

Feature	Ridge Coefficient
Appliances	3.6817
Usage Hours	1.0696
House Size	0.2443
Occupants	2.9836

All four features have positive coefficients, indicating that higher values of these features are associated with higher electricity consumption.

j) Explain how Ridge Regression reduces overfitting.

Ridge Regression reduces overfitting by adding a **penalty for large coefficients** to the Linear Regression model.

The Ridge objective function is:

Minimize Error + $\alpha \sum \text{Coefficient}^2$

The penalty forces the model coefficients to become smaller without normally making them exactly zero.

Advantages of Ridge Regression

1. **Reduces overfitting** by controlling the size of coefficients.
2. **Handles multicollinearity** when input features are highly correlated.
3. Produces a more stable model.
4. Prevents the model from depending too heavily on any single feature.
5. Improves generalization to unseen data.

Conclusion

Ridge Regression is suitable for electricity consumption prediction because features such as **number of appliances and usage hours may be correlated**. With $\alpha = 1.0$, the model controls the coefficients and reduces the risk of overfitting.

The model achieved an **R² score of approximately 0.8092**, and the predicted electricity consumption for the new house is approximately **212.73 units**

Question 5: Delivery Time Prediction Using Linear Regression

A logistics company wants to predict **delivery time** based on distance, traffic level, number of packages, and weather score.

Delivery	Distance km	Traffic Level	Packages	Weather Score	Time Hours
D1	10	2	20	8	1.2
D2	20	4	30	7	2.4
D3	30	6	40	6	3.5
D4	40	8	50	5	4.8
D5	50	9	60	4	6.0
D6	15	3	25	8	1.8
D7	35	7	45	5	4.2
D8	25	5	35	6	3.0

Tasks

a) Create the dataset.

```
import pandas as pd
```

```
data = {
```

```
    "Delivery": ["D1", "D2", "D3", "D4", "D5", "D6", "D7", "D8"],
```

```
    "Distance": [10, 20, 30, 40, 50, 15, 35, 25],
```

```
"Traffic": [2, 4, 6, 8, 9, 3, 7, 5],  
"Packages": [20, 30, 40, 50, 60, 25, 45, 35],  
"Weather": [8, 7, 6, 5, 4, 8, 5, 6],  
"Time": [1.2, 2.4, 3.5, 4.8, 6.0, 1.8, 4.2, 3.0]  
}
```

```
df = pd.DataFrame(data)  
  
print("Delivery Time Dataset:")  
  
print(df)
```

b) Identify Input and Output Variables

Input Variables

The independent variables used to predict delivery time are:

1. **Distance (km)**
2. **Traffic Level**
3. **Packages**
4. **Weather Score**

Therefore:

```
X=[Distance, Traffic, Packages, Weather]
```

Output Variable

The dependent variable is:

- **Time (Hours)**

Therefore:

```
Y=Time
```

Python Code

```
X = df[["Distance", "Traffic", "Packages", "Weather"]]
```

```
y = df["Time"]
```

```
print("Input Variables:")
```

```
print(X)
```

```
print("\nOutput Variable:")
```

```
print(y)
```

c) Split the Data into Train and Test Sets

We use **75% for training** and **25% for testing**.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.25,
```

```
    random_state=42
```

```
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

Since there are 8 delivery records:

- **Training data = 6 records**
- **Testing data = 2 records**

d) Implement Linear Regression

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```



```
print("Linear Regression model trained successfully.")
```

e) Predict Delivery Time for Test Records

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Delivery Time:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual Time:", actual, "Hours")
```

```
    print("Predicted Time:", round(predicted, 2), "Hours")
```

```
    print()
```

Output

Delivery	Actual Time (Hours)	Predicted Time (Hours)
----------	---------------------	------------------------

D2	2.4	2.37
----	-----	------

D6	1.8	1.74
----	-----	------

The predicted values are close to the actual delivery times.

f) Predict Delivery Time for a New Delivery

New delivery details:

Distance (km)	Traffic Level	Packages	Weather Score
---------------	---------------	----------	---------------

28	6	38	6
----	---	----	---

Python Code

```
new_delivery = pd.DataFrame({
```

```
    "Distance": [28],
```

```
    "Traffic": [6],
```

```
    "Packages": [38],
```

```
    "Weather": [6]
```

```
})

new_time = model.predict(new_delivery)

print("New Delivery Details:")

print(new_delivery)

print("\nPredicted Delivery Time:")

print(round(new_time[0], 2), "Hours")
```

Output

Predicted Delivery Time:

3.32 Hours

Therefore, the predicted delivery time is approximately **3.32 hours**.

g) Evaluate the Model

The model is evaluated using:

- **MAE**
- **MSE**
- **RMSE**
- **R² Score**

Python Code

```
from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Model Evaluation:")

print("MAE :", round(mae, 4))
```

```
print("MSE :", round(mse, 4))  
print("RMSE :", round(rmse, 4))  
print("R2 Score:", round(r2, 4))
```

Output

Metric	Value
MAE	0.0452 Hours
MSE	0.0023
RMSE	0.0480 Hours
R ² Score	0.9744

Interpretation

- **MAE = 0.0452 hours:** The average prediction error is approximately **0.045 hours**, or about **2.7 minutes**.
- **MSE = 0.0023:** The squared prediction error is very small.
- **RMSE = 0.0480 hours:** The typical prediction error is approximately **0.048 hours**, or about **2.9 minutes**.
- **R² = 0.9744:** The model explains approximately **97.44% of the variation in delivery time**.

Therefore, the Linear Regression model performs well on this dataset.

h) How the Model Helps a Logistics Company

The Linear Regression model can help a logistics company **estimate delivery times before assigning or scheduling deliveries**.

For example, the company can enter:

- Distance
- Traffic level
- Number of packages
- Weather conditions

and the model will estimate the expected delivery time.

Benefits

1. **Better scheduling:** The company can estimate when deliveries are likely to arrive.
2. **Route planning:** Longer distances and higher traffic can be considered when planning deliveries.
3. **Customer communication:** Customers can receive more accurate estimated delivery times.
4. **Resource allocation:** Drivers and delivery vehicles can be assigned more efficiently.
5. **Delay management:** The company can identify deliveries likely to take longer and take preventive action.
6. **Operational efficiency:** Better delivery-time estimates can reduce waiting time and improve overall logistics operations.

Conclusion

The Linear Regression model achieved an **R² score of 0.9744**, indicating a strong relationship between the given delivery factors and delivery time in this small dataset. For the new delivery with **28 km distance, traffic level 6, 38 packages, and weather score 6**, the predicted delivery time is approximately **3.32 hours**.

Thus, the model can be useful as a basic tool for **delivery scheduling and logistics planning**, although a larger real-world dataset would be needed for reliable production use.

Question 6: Loan Approval Prediction Using Logistic Regression

A bank wants to predict whether a loan application should be **approved** or **rejected**.

Applicant	Income	Credit Score	Existing Loan	Employment Years	Approved
A1	25000	580	200000	1	0
A2	30000	600	180000	2	0
A3	45000	680	120000	3	1
A4	60000	750	80000	5	1
A5	70000	800	50000	6	1
A6	28000	590	220000	1	0
A7	55000	720	100000	4	1
A8	35000	620	160000	2	0

Here,

Approved = 1 means loan approved.

Approved = 0 means loan rejected.

Tasks

a) Create the Dataset in Python

```
import pandas as pd
```

```
data = {  
    "Applicant": ["A1", "A2", "A3", "A4", "A5", "A6", "A7", "A8"],  
    "Applicant_Income": [25000, 30000, 45000, 60000, 70000, 28000, 55000, 35000],  
    "Credit_Score": [580, 600, 680, 750, 800, 590, 720, 620],  
    "Existing_Loan": [200000, 180000, 120000, 80000, 50000, 220000, 100000, 160000],  
    "Employment_Years": [1, 2, 3, 5, 6, 1, 4, 2],  
    "Approved": [0, 0, 1, 1, 1, 0, 1, 0]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Loan Approval Dataset:")
```

```
print(df)
```

b) Identify Independent and Dependent Variables

Independent Variables

The features used to predict loan approval are:

1. **Applicant Income**
2. **Credit Score**
3. **Existing Loan**
4. **Employment Years**

Therefore:

```
X=[Income, Credit Score, Existing Loan, Employment Years]
```

Dependent Variable

The target variable is:

- **Approved**

Where:

- **Approved = 1** → Loan approved
- **Approved = 0** → Loan rejected

Therefore:

Y=Approved

Python Code

```
X = df[[
    "Applicant_Income",
    "Credit_Score",
    "Existing_Loan",
    "Employment_Years"
]]
y = df["Approved"]
print("Independent Variables:")
print(X)
print("\nDependent Variable:")
print(y)
```

c) Split the Dataset

We split the data into **75% training** and **25% testing** data.

Since the dataset is very small, **stratify=y** is used so that both **approved and rejected** applications are represented in the test data.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
```

```
test_size=0.25,  
random_state=42,  
stratify=y  
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

There are 8 applicants:

- **Training data = 6 applicants**
- **Testing data = 2 applicants**

The test records are:

Applicant	Actual Approved
------------------	------------------------

A6	0
----	---

A5	1
----	---

d) Implement Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Logistic Regression model trained successfully.")
```

Logistic Regression is used because the target variable has two classes:

0 = Rejected
1 = Approved

e) Predict Approval Status for Test Data

```
y_pred = model.predict(X_test)

print("Actual vs Predicted Approval:")

for actual, predicted in zip(y_test, y_pred):

    print("Actual:", actual, "Predicted:", predicted)
```

Output

Applicant	Actual	Predicted
------------------	---------------	------------------

A6	0	0
----	---	---

A5	1	1
----	---	---

Therefore:

- **A6 → Loan Rejected**
- **A5 → Loan Approved**

f) Predict Loan Approval for a New Applicant

New applicant details:

Income	Credit Score	Existing Loan	Employment Years
---------------	---------------------	----------------------	-------------------------

50000	700	100000	4
-------	-----	--------	---

Python Code

```
new_applicant = pd.DataFrame({

    "Applicant_Income": [50000],

    "Credit_Score": [700],

    "Existing_Loan": [100000],

    "Employment_Years": [4]
```



```
})
```

```
new_prediction = model.predict(new_applicant)
```

```
print("New Applicant:")
```

```
print(new_applicant)
```

```
print("\nPredicted Loan Status:")
```

```
if new_prediction[0] == 1:
```

```
    print("Loan APPROVED")
```

```
else:
```

```
    print("Loan REJECTED")
```

Output

Predicted Loan Status:

Loan APPROVED

Therefore, the new applicant is predicted to have:

Approved = 1 → Loan Approved

g) Evaluate the Model

The model is evaluated using:

- Accuracy
- Confusion Matrix
- Precision
- Recall
- F1-score

Python Code

```
from sklearn.metrics import (
```

```
    accuracy_score,
```

```
    confusion_matrix,
```

```
    precision_score,
```

```
    recall_score,
```

```
f1_score
)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
print("Model Evaluation:")
print("Accuracy :", accuracy)
print("\nConfusion Matrix:")
print(cm)
print("\nPrecision:", precision)
print("Recall  :", recall)
print("F1 Score :", f1)
```

Output

Model Evaluation:

Accuracy : 1.0

Confusion Matrix:

[[1 0]

[0 1]]

Precision: 1.0

Recall : 1.0

F1 Score : 1.0

Interpretation

Metric	Value	Meaning
Accuracy	100%	All test applications were correctly classified
Precision	100%	All predicted approvals were actually approved
Recall	100%	All actual approved applications were correctly identified
F1-score	100%	Excellent balance between precision and recall

Confusion Matrix

	Predicted	
	Rejected	Approved
Actual Rejected	1	0
Actual Approved	0	1

So there are:

- **True Negative = 1**
- **True Positive = 1**
- **False Positive = 0**
- **False Negative = 0**

Note: The 100% scores should not be treated as proof of real-world performance because the dataset contains only **8 applicants**. A much larger dataset would be needed for a reliable loan approval system.

h) Why Logistic Regression is Suitable for Loan Approval Prediction

Logistic Regression is suitable because **loan approval is a binary classification problem**.

There are only two possible outcomes:

- **0 → Loan Rejected**
- **1 → Loan Approved**

The model can use applicant information such as:

- Applicant income
- Credit score
- Existing loan amount
- Employment experience

to estimate the probability that a loan application should be approved.

Advantages

1. **Suitable for binary classification** such as approved/rejected.
2. **Simple and easy to interpret.**
3. Produces a **probability of approval.**
4. Can identify relationships between applicant characteristics and approval decisions.
5. Works efficiently with structured numerical data.

Conclusion

Logistic Regression is an appropriate model for predicting loan approval because the target variable has two classes: **approved and rejected**. For the given dataset, the model correctly classified the test applications and achieved **100% accuracy, precision, recall, and F1-score**. The new applicant with an income of **₹50,000**, credit score of **700**, existing loan of **₹1,00,000**, and **4 years of employment** is predicted to have the loan **approved**.

Question 7: Healthcare Treatment Cost Prediction Using Lasso Regression

A hospital wants to predict **treatment cost** based on patient health parameters. Since many medical features are available, the hospital wants to use Lasso Regression to identify important predictors.

Patient	Age	BP	Sugar Level	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000
P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

Tasks

a) Create the Dataset Using Python

```
import pandas as pd
```

```
data = {  
    "Patient": ["P1", "P2", "P3", "P4", "P5", "P6", "P7", "P8"],  
    "Age": [25, 35, 45, 55, 60, 30, 50, 40],  
    "BP": [115, 125, 140, 155, 165, 120, 150, 135],  
    "Sugar": [90, 110, 150, 180, 200, 100, 170, 130],  
    "BMI": [22, 25, 29, 32, 35, 24, 31, 27],  
    "Previous_Visits": [1, 2, 3, 5, 6, 1, 4, 2],  
    "Treatment_Cost": [5000, 8000, 15000, 25000, 32000, 7000, 22000, 12000]  
}
```

```
df = pd.DataFrame(data)  
  
print("Healthcare Treatment Cost Dataset:")  
  
print(df)
```

Output

Patient	Age	BP	Sugar	BMI	Previous Visits	Treatment Cost
P1	25	115	90	22	1	5000
P2	35	125	110	25	2	8000
P3	45	140	150	29	3	15000
P4	55	155	180	32	5	25000
P5	60	165	200	35	6	32000
P6	30	120	100	24	1	7000

P7	50	150	170	31	4	22000
P8	40	135	130	27	2	12000

b) Identify Features and Target Variable

Independent Variables / Features

The features used to predict treatment cost are:

1. Age
2. BP
3. Sugar Level
4. BMI
5. Previous Visits

Therefore,

$X = [\text{Age}, \text{BP}, \text{Sugar}, \text{BMI}, \text{Previous Visits}]$

Dependent Variable / Target

The target variable is:

Treatment Cost

$Y = \text{Treatment Cost}$

Python Code

```
X = df[["Age", "BP", "Sugar", "BMI", "Previous_Visits"]]
```

```
y = df["Treatment_Cost"]
```

```
print("Features:")
```

```
print(X)
```

```
print("\nTarget Variable:")
```

```
print(y)
```

c) Split into Training and Testing Sets

We use 75% for training and 25% for testing.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.25,  
    random_state=42  
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

Since there are 8 patients:

- Training data = 6 patients
- Testing data = 2 patients

With **random_state=42**, the test patients are P2 and P6.

d) Implement Lasso Regression

Lasso Regression is a regularized regression technique that can reduce the effect of less important features.

```
from sklearn.linear_model import Lasso
```

```
model = Lasso(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
print("Lasso Regression model trained successfully.")
```

e) Try Alpha Values: 0.1, 1.0 and 10.0

We can compare different values of alpha.

```
from sklearn.linear_model import Lasso
```

```
alphas = [0.1, 1.0, 10.0]
```

```
for alpha in alphas:
```

```
    model = Lasso(alpha=alpha, max_iter=100000)
```

```
    model.fit(X_train, y_train)
```

```
    print("\nAlpha =", alpha)
```

```
    print("Coefficients:")
```

```
    for feature, coefficient in zip(X.columns, model.coef_):
```

```
        print(feature, ":", round(coefficient, 4))
```

Coefficient Comparison

Feature	$\alpha = 0.1$	$\alpha = 1.0$	$\alpha = 10.0$
Age	-753.42	-757.03	-800.36
BP	816.36	819.10	803.27
Sugar	-23.26	-21.00	2.32
BMI	-13.71	-26.88	0.00
Previous Visits	3061.50	3043.67	2921.28

For this dataset, $\alpha = 10.0$ reduces the BMI coefficient to exactly zero.

Note: Lasso coefficients can look unusual here because the dataset is extremely small and the medical features are strongly correlated. For a real analysis, feature scaling and a much larger dataset would be appropriate.

f) Predict Treatment Cost for Test Patients

Using $\alpha = 1.0$:

```
model = Lasso(alpha=1.0, max_iter=100000)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Treatment Cost:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual Cost:", actual)
```

```
    print("Predicted Cost:", round(predicted, 2))
```

```
    print()
```

Output

Patient	Actual Cost	Predicted Cost
P2	₹8,000	₹8,145.10
P6	₹7,000	₹5,027.91

The model predicts the treatment costs using the patient's medical features.

g) Predict Treatment Cost for a New Patient

New patient:

Age	BP	Sugar Level	BMI	Previous Visits
48	145	160	30	4

Python Code

```
# New patient data

new_patient = pd.DataFrame({

    "Age": [48],

    "BP": [145],

    "Sugar": [160],

    "BMI": [30],

    "Previous_Visits": [4]

})

new_cost = model.predict(new_patient)

print("New Patient:")

print(new_patient)

print("\nPredicted Treatment Cost:")

print(round(new_cost[0], 2))
```

Output

Predicted Treatment Cost:

19588.88

Therefore, the predicted treatment cost is approximately:

₹19,588.88

or approximately ₹19,589.

h) Evaluate the Model

The model is evaluated using:

- MAE
- MSE
- RMSE
- R^2 Score

```
from sklearn.metrics import mean_absolute_error
```

```
from sklearn.metrics import mean_squared_error
```

```

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Model Evaluation:")

print("MAE :", round(mae, 2))

print("MSE :", round(mse, 2))

print("RMSE :", round(rmse, 2))

print("R2 Score:", round(r2, 4))

```

Output for $\alpha = 1.0$

Metric	Value
MAE	1058.60
MSE	1,955,101.27
RMSE	1398.25
R ² Score	-6.8204

Interpretation

- MAE = ₹1,058.60: On average, the predictions differ from the actual treatment cost by about ₹1,059.
- MSE = 1,955,101.27: Represents the average squared prediction error.
- RMSE = ₹1,398.25: The typical prediction error is approximately ₹1,398.
- R² = -6.8204: The model performs poorly on this particular two-patient test set.

The negative R² is mainly because the dataset contains only 8 patients, resulting in an extremely small test set of only 2 patients. Therefore, the evaluation score is not reliable enough to judge real-world performance.

i) Print the Model Coefficients

For $\alpha = 1.0$:

```
print("Lasso Regression Coefficients:")
```

```
for feature, coefficient in zip(X.columns, model.coef_):
```

```
    print(feature, ":", round(coefficient, 4))
```

```
print("\nIntercept:", round(model.intercept_, 4))
```

Output

Lasso Regression Coefficients:

Age : -757.0283

BP : 819.1026

Sugar : -20.9973

BMI : -26.8783

Previous_Visits : 3043.6743

Intercept: -70852.4250

j) Important Medical Features Based on Lasso Coefficients

Lasso identifies important features by shrinking their coefficients toward zero.

For $\alpha = 1.0$:

Feature	Coefficient	Interpretation
Age	-757.03	Influences treatment cost

BP	819.10	Influences treatment cost
Sugar Level	-21.00	Smaller influence
BMI	-26.88	Smaller influence
Previous Visits	3043.67	Strong influence

In this model, Previous Visits has the largest coefficient magnitude, followed by BP and Age.

For $\alpha = 10.0$, Lasso reduces the BMI coefficient to 0, meaning BMI is effectively removed from the model at that regularization strength.

Conclusion

Lasso Regression is useful for healthcare cost prediction because it can both predict treatment cost and perform feature selection. By increasing the alpha value, Lasso applies stronger regularization and can reduce less important features toward zero. In this dataset, BMI is reduced to zero when $\alpha = 10.0$.

However, because the dataset contains only 8 patients, the model evaluation is not reliable for real-world healthcare decisions. A larger dataset with more patients and additional clinical features would be required for a practical treatment-cost prediction system.

Question 8: Manufacturing Defect Strength Prediction Using Ridge Regression

A manufacturing company wants to predict the **strength score** of a product based on machine parameters. Since temperature, pressure, and machine speed may be correlated, Ridge Regression is used.

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76
P7	78	36	115	7	81
P8	88	44	128	5	80

Tasks

a) Create the dataset.

```
import pandas as pd
```

```
data = {  
    "Product": ["P1", "P2", "P3", "P4", "P5", "P6", "P7", "P8"],  
    "Temperature": [70, 75, 80, 85, 90, 95, 78, 88],  
    "Pressure": [30, 35, 38, 42, 45, 50, 36, 44],  
    "Machine_Speed": [100, 110, 120, 125, 130, 140, 115, 128],  
    "Material_Quality": [8, 7, 6, 6, 5, 4, 7, 5],  
    "Strength_Score": [75, 78, 80, 82, 79, 76, 81, 80]  
}
```

```
df = pd.DataFrame(data)
```

```
print("Manufacturing Dataset:")
```

```
print(df)
```

Dataset

Product	Temperature	Pressure	Machine Speed	Material Quality	Strength Score
P1	70	30	100	8	75
P2	75	35	110	7	78
P3	80	38	120	6	80
P4	85	42	125	6	82
P5	90	45	130	5	79
P6	95	50	140	4	76

P7	78	36	115	7	81
P8	88	44	128	5	80

b) Identify Independent and Dependent Variables

Independent Variables

The features used to predict the strength score are:

1. **Temperature**
2. **Pressure**
3. **Machine Speed**
4. **Material Quality**

Therefore:

$X = [\text{Temperature, Pressure, Machine Speed, Material Quality}]$

Dependent Variable

The target variable is:

Strength Score

$Y = \text{Strength Score}$

Python Code

```
X = df[["Temperature",
        "Pressure",
        "Machine_Speed",
        "Material_Quality"]]

y = df["Strength_Score"]

print("Independent Variables:")

print(X)
```

```
print("\nDependent Variable:")
```

```
print(y)
```

c) Split into Training and Testing Data

We use **75% for training** and **25% for testing**.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.25,
```

```
    random_state=42
```

```
)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

Since there are 8 products:

- **Training data = 6 products**
- **Testing data = 2 products**

With **random_state=42**, the test products are **P2 and P6**.

d) Implement Ridge Regression

Ridge Regression is a regularized version of Linear Regression. It is useful when independent variables are highly correlated.

```
from sklearn.linear_model import Ridge
```

```
model = Ridge(alpha=1.0)
```

```
model.fit(X_train, y_train)
```

```
print("Ridge Regression model trained successfully.")
```


e) Use Alpha Values 0.1, 1.0 and 10.0

We can compare the performance of Ridge Regression for different alpha values.

```
from sklearn.linear_model import Ridge

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import numpy as np

alphas = [0.1, 1.0, 10.0]

for alpha in alphas:

    model = Ridge(alpha=alpha)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    mae = mean_absolute_error(y_test, y_pred)

    mse = mean_squared_error(y_test, y_pred)

    rmse = np.sqrt(mse)

    r2 = r2_score(y_test, y_pred)

    print("\nAlpha =", alpha)

    print("MAE :", round(mae, 4))

    print("MSE :", round(mse, 4))

    print("RMSE:", round(rmse, 4))

    print("R2 Score:", round(r2, 4))
```

Alpha Comparison

Alpha	MAE	MSE	RMSE	R ²
0.1	3.5512	22.8999	4.7854	-21.8999
1.0	3.3354	21.5103	4.6379	-20.5103

10.0 3.2834 20.8738 4.5688 -19.8738

For this particular train-test split, $\alpha = 10.0$ gives the lowest MAE, MSE, and RMSE.

However, the R^2 values are negative because the test set contains only **2 products**, so the evaluation is highly unstable.

f) Predict Strength Score for Test Products

For the main model, we use $\alpha = 1.0$.

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Actual vs Predicted Strength Score:")

for actual, predicted in zip(y_test, y_pred):

    print("Actual Strength:", actual)

    print("Predicted Strength:", round(predicted, 2))

    print()
```

Output

Product	Actual Strength	Predicted Strength
P2	78	78.11
P6	76	82.56

The model predicts P2 quite closely, while the prediction for P6 has a larger error.

g) Predict Strength Score for a New Product

New product details:

Temperature	Pressure	Machine Speed	Material Quality
82	40	122	6

Python Code

```
new_product = pd.DataFrame({
    "Temperature": [82],
    "Pressure": [40],
    "Machine_Speed": [122],
    "Material_Quality": [6]
})

new_strength = model.predict(new_product)

print("New Product Details:")

print(new_product)

print("\nPredicted Strength Score:")

print(round(new_strength[0], 2))
```

Output

Predicted Strength Score:

80.89

Therefore, the predicted strength score of the new product is approximately:

80.89

h) Evaluate Using MAE, MSE, RMSE and R²

For $\alpha = 1.0$:

```
from sklearn.metrics import mean_absolute_error

from sklearn.metrics import mean_squared_error
```

```

from sklearn.metrics import r2_score

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

```

```

print("Model Evaluation:")

print("MAE :", round(mae, 4))

print("MSE :", round(mse, 4))

print("RMSE :", round(rmse, 4))

print("R2 Score:", round(r2, 4))

```

Output for $\alpha = 1.0$

Metric	Value
MAE	3.3354
MSE	21.5103
RMSE	4.6379
R ² Score	-20.5103

Interpretation

- **MAE = 3.3354:** The average prediction error is approximately **3.34 strength-score points**.
- **MSE = 21.5103:** Represents the average squared prediction error.
- **RMSE = 4.6379:** The typical prediction error is approximately **4.64 strength-score points**.
- **R² = -20.5103:** The model performs poorly on this extremely small test set.

Important: The negative R^2 does **not necessarily mean Ridge Regression is unsuitable**. There are only **8 observations**, and the test set contains only **2 products**. R^2 calculated from two test observations is highly unstable.

i) Display Ridge Coefficients

For $\alpha = 1.0$:

```
print("Ridge Regression Coefficients:")

for feature, coefficient in zip(X.columns, model.coef_):

    print(feature, ":", round(coefficient, 4))

print("\nIntercept:", round(model.intercept_, 4))
```

Output

Ridge Regression Coefficients:

Temperature : -0.6724

Pressure : -0.1134

Machine_Speed : 0.7587

Material_Quality : 1.0556

Intercept: 41.6650

Coefficient Table

Feature	Ridge Coefficient
Temperature	-0.6724
Pressure	-0.1134
Machine Speed	0.7587

Material Quality 1.0556

The coefficients indicate the direction and relative effect of each feature while the other variables are held constant.

j) How Ridge Handles Multicollinearity

In manufacturing data, variables such as **temperature, pressure, and machine speed** may be highly correlated. This is called **multicollinearity**.

When multicollinearity exists, ordinary Linear Regression can produce unstable coefficients. Small changes in the data can cause large changes in the coefficients.

Ridge Regression solves this problem by adding an **L2 regularization penalty**:

Minimize Error + $\alpha \sum \beta_i^2$

This penalty forces the coefficients to become smaller and more stable.

How Ridge helps

1. **Reduces coefficient size** by applying an L2 penalty.
2. **Handles correlated features** more effectively than ordinary Linear Regression.
3. **Reduces overfitting**.
4. **Improves model stability**.
5. Keeps correlated features in the model rather than completely removing them.

Conclusion

Ridge Regression is suitable for manufacturing strength prediction because machine parameters such as **temperature, pressure, and machine speed can be correlated**. Ridge reduces the effect of this multicollinearity by shrinking the regression coefficients.

For $\alpha = 1.0$, the predicted strength score for the new product is approximately **80.89**.

However, the dataset contains only **8 products**, so the evaluation results—especially the negative R^2 —should not be used to judge real-world performance. A larger manufacturing dataset would be needed for a reliable prediction system.

Question 9: Student Salary Package Prediction Using Linear Regression

A university placement cell wants to predict the **expected salary package** of students.

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package LPA
S1	6.5	60	55	65	3.2
S2	7.0	65	60	70	4.0

Student	CGPA	Aptitude Score	Coding Score	Communication Score	Package LPA
S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {
    "Student": ["S1", "S2", "S3", "S4", "S5", "S6", "S7", "S8"],
    "CGPA": [6.5, 7.0, 7.5, 8.0, 8.5, 9.0, 6.8, 8.2],
    "Aptitude_Score": [60, 65, 70, 78, 85, 90, 62, 80],
    "Coding_Score": [55, 60, 68, 80, 88, 92, 58, 82],
    "Communication_Score": [65, 70, 72, 75, 80, 85, 68, 78],
    "Package_LPA": [3.2, 4.0, 5.0, 6.5, 8.0, 10.0, 3.8, 7.0]
}
```

```
df = pd.DataFrame(data)
```

```
print("Student Salary Package Dataset:")
```

```
print(df)
```

Dataset

Student	CGPA	Aptitude	Coding	Communication	Package (LPA)
---------	------	----------	--------	---------------	---------------

S1	6.5	60	55	65	3.2
----	-----	----	----	----	-----

S2	7.0	65	60	70	4.0
----	-----	----	----	----	-----

S3	7.5	70	68	72	5.0
S4	8.0	78	80	75	6.5
S5	8.5	85	88	80	8.0
S6	9.0	90	92	85	10.0
S7	6.8	62	58	68	3.8
S8	8.2	80	82	78	7.0

b) Identify Input and Output Variables

Input Variables

The independent variables used to predict the salary package are:

1. **CGPA**
2. **Aptitude Score**
3. **Coding Score**
4. **Communication Score**

Therefore:

$X = [\text{CGPA}, \text{Aptitude}, \text{Coding}, \text{Communication}]$

Output Variable

The dependent variable is:

Package (LPA)

$Y = \text{Package}$

Python Code

```
X = df[[
    "CGPA",
    "Aptitude_Score",
    "Coding_Score",
```



```
"Communication_Score"

]]

y = df["Package_LPA"]

print("Input Variables:")

print(X)

print("\nOutput Variable:")

print(y)
```

c) Split the Dataset

We divide the dataset into **75% training** and **25% testing** data.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.25,

    random_state=42

)

print("Training Data:")

print(X_train)

print("\nTesting Data:")

print(X_test)
```

Since there are 8 students:

- **Training data = 6 students**
- **Testing data = 2 students**

With **random_state=42**, the test students are **S2 and S6**.

d) Implement Linear Regression

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Linear Regression model trained successfully.")
```

The model learns the relationship between the students' academic/skill scores and their expected salary package.

e) Predict Package for Test Students

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Package:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual Package:", actual, "LPA")
```

```
    print("Predicted Package:", round(predicted, 2), "LPA")
```

```
    print()
```

Output

Student	Actual Package	Predicted Package
----------------	-----------------------	--------------------------

S2	4.0 LPA	3.91 LPA
----	---------	----------

S6	10.0 LPA	9.55 LPA
----	----------	----------

The predicted values are reasonably close to the actual package values.

f) Predict Package for a New Student

New student details:

CGPA	Aptitude Score	Coding Score	Communication Score
-------------	-----------------------	---------------------	----------------------------

8.1	82	85	78
-----	----	----	----

Python Code

```
new_student = pd.DataFrame({  
    "CGPA": [8.1],  
    "Aptitude_Score": [82],  
    "Coding_Score": [85],  
    "Communication_Score": [78]  
})  
  
new_package = model.predict(new_student)  
  
print("New Student Details:")  
  
print(new_student)  
  
print("\nPredicted Salary Package:")  
  
print(round(new_package[0], 2), "LPA")
```

Output

Predicted Salary Package:

6.99 LPA

Therefore, the predicted salary package for the new student is approximately:

6.99 LPA

g) Evaluate the Model

The model is evaluated using:

- **MAE**
- **MSE**
- **RMSE**
- **R² Score**

Python Code

```
from sklearn.metrics import mean_absolute_error  
  
from sklearn.metrics import mean_squared_error  
  
from sklearn.metrics import r2_score
```

```

import numpy as np

mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("Model Evaluation:")

print("MAE :", round(mae, 4))

print("MSE :", round(mse, 4))

print("RMSE :", round(rmse, 4))

print("R2 Score:", round(r2, 4))

```

Output

Metric	Value
MAE	0.27 LPA
MSE	0.0736
RMSE	0.27 LPA
R ² Score	0.9880

Interpretation

- **MAE = 0.27 LPA:** On average, the predicted package differs from the actual package by approximately **0.27 LPA**.
- **MSE = 0.0736:** Represents the average squared prediction error.
- **RMSE = 0.27 LPA:** The typical prediction error is approximately **0.27 LPA**.
- **R² = 0.9880:** The model explains approximately **98.8% of the variation in salary packages** in the test data.

Therefore, the model performs well on this small dataset.

h) Interpretation from the Placement Cell Perspective

The Linear Regression model can help the university placement cell **estimate the expected salary package of students** based on their CGPA, aptitude, coding, and communication skills.

For example, the new student with:

- **CGPA = 8.1**
- **Aptitude = 82**
- **Coding = 85**
- **Communication = 78**

has a predicted package of approximately **6.99 LPA**.

Benefits for the Placement Cell

1. **Salary estimation:** The placement cell can estimate the expected package of students before placements.
2. **Student preparation:** Students with lower predicted packages can be given additional training.
3. **Skill identification:** The college can identify areas such as coding, aptitude, or communication that need improvement.
4. **Placement planning:** The placement team can plan training programs based on predicted outcomes.
5. **Career guidance:** Students can use predictions to understand their potential placement opportunities.

Conclusion

The Linear Regression model achieved an **R² score of approximately 0.988**, indicating a strong relationship between student performance and salary package in this small dataset. The model predicts approximately **6.99 LPA** for the new student.

However, the dataset contains only **8 students**, so the results should be treated as an academic demonstration rather than a reliable real-world salary prediction system. A larger dataset containing factors such as internship experience, certifications, projects, college, company, and interview performance would improve the model.

Question 10: Employee Attrition Prediction Using Logistic Regression

An HR department wants to predict whether an employee will **leave** the company or **stay**.

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1
E4	5	6	6	8	0

Employee	Experience	Satisfaction Score	Overtime Hours	Salary Increment %	Leave
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0
E8	7	4	11	5	1

Here,

Leave = 1 means employee may leave.

Leave = 0 means employee may stay.

Tasks

a) Create the dataset using Python.

```
import pandas as pd
```

```
data = {
```

```
    "Employee": ["E1", "E2", "E3", "E4", "E5", "E6", "E7", "E8"],
```

```
    "Experience": [2, 3, 1, 5, 6, 8, 4, 7],
```

```
    "Satisfaction_Score": [8, 7, 4, 6, 3, 2, 7, 4],
```

```
    "Overtime_Hours": [2, 4, 10, 6, 12, 14, 3, 11],
```

```
    "Salary_Increment": [12, 10, 5, 8, 4, 3, 11, 5],
```

```
    "Leave": [0, 0, 1, 0, 1, 1, 0, 1]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Employee Attrition Dataset:")
```

```
print(df)
```

Dataset

Employee	Experience	Satisfaction	Overtime	Salary Increment	Leave
E1	2	8	2	12	0
E2	3	7	4	10	0
E3	1	4	10	5	1

E4	5	6	6	8	0
E5	6	3	12	4	1
E6	8	2	14	3	1
E7	4	7	3	11	0
E8	7	4	11	5	1

b) Identify Features and Target Variable

Independent Variables / Features

The features used to predict employee attrition are:

1. **Experience**
2. **Satisfaction Score**
3. **Overtime Hours**
4. **Salary Increment %**

Therefore:

$X = [\text{Experience, Satisfaction, Overtime, Salary Increment}]$

Dependent Variable / Target

The target variable is:

Leave

- **Leave = 1** → Employee may leave
- **Leave = 0** → Employee may stay

$Y = \text{Leave}$

Python Code

```
X = df[[
    "Experience",
    "Satisfaction_Score",
    "Overtime_Hours",
```

```
"Salary_Increment"

]]

y = df["Leave"]

print("Features:")

print(X)

print("\nTarget Variable:")

print(y)
```

c) Split into Training and Testing Data

We use **75% for training** and **25% for testing**.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(

    X,

    y,

    test_size=0.25,

    random_state=42,

    stratify=y

)
```

```
print("Training Data:")

print(X_train)

print("\nTesting Data:")

print(X_test)
```

Since there are 8 employees:

- **Training data = 6 employees**
- **Testing data = 2 employees**

d) Implement Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
print("Logistic Regression model trained successfully.")
```

Logistic Regression is appropriate because **Leave** has two possible outcomes: **0 or 1**.

e) Predict Attrition for Test Employees

```
y_pred = model.predict(X_test)
```

```
print("Actual vs Predicted Attrition:")
```

```
for actual, predicted in zip(y_test, y_pred):
```

```
    print("Actual:", actual, "Predicted:", predicted)
```

Output

Employee	Actual Leave	Predicted Leave
----------	--------------	-----------------

E2	0	0
----	---	---

E6	1	1
----	---	---

Therefore:

- **E2 → Stay**
- **E6 → Leave**

f) Predict Attrition for a New Employee

New employee details:

Experience	Satisfaction Score	Overtime Hours	Salary Increment %
------------	--------------------	----------------	--------------------

5	4	9	6
---	---	---	---

Python Code

```
new_employee = pd.DataFrame({  
    "Experience": [5],  
    "Satisfaction_Score": [4],  
    "Overtime_Hours": [9],  
    "Salary_Increment": [6]  
})  
  
prediction = model.predict(new_employee)  
  
print("New Employee:")  
  
print(new_employee)  
  
print("\nPredicted Attrition:")  
  
if prediction[0] == 1:  
    print("Employee may LEAVE")  
else:  
    print("Employee may STAY")
```

Output

Predicted Attrition:

Employee may LEAVE

Therefore:

Leave = 1 → Employee may leave the company.

We can also calculate the probability:

```
probability = model.predict_proba(new_employee)  
  
print("Probability of Staying:", round(probability[0][0] * 100, 2), "%")  
  
print("Probability of Leaving:", round(probability[0][1] * 100, 2), "%")
```

g) Evaluate the Model

The model is evaluated using:

- **Accuracy**
- **Confusion Matrix**
- **Precision**
- **Recall**
- **F1-score**

Python Code

```
from sklearn.metrics import (  
  
    accuracy_score,  
  
    confusion_matrix,  
  
    precision_score,  
  
    recall_score,  
  
    f1_score  
)  
  
metricsaccuracy = accuracy_score(y_test, y_pred)  
  
cm = confusion_matrix(y_test, y_pred)  
  
precision = precision_score(y_test, y_pred)  
  
recall = recall_score(y_test, y_pred)  
  
f1 = f1_score(y_test, y_pred)  
  
print("Model Evaluation:")  
  
print("Accuracy :", round(accuracy, 4))  
  
  
print("\nConfusion Matrix:")  
  
print(cm)  
  
  
print("\nPrecision:", round(precision, 4))  
  
print("Recall  :", round(recall, 4))
```

```
print("F1 Score :", round(f1, 4))
```

Output

Metric	Value
--------	-------

Accuracy	1.00 (100%)
----------	-------------

Precision	1.00 (100%)
-----------	-------------

Recall	1.00 (100%)
--------	-------------

F1-score	1.00 (100%)
----------	-------------

Confusion Matrix

Predicted

Stay Leave

Actual Stay 1 0

Actual Leave 0 1

Therefore:

- **True Negative = 1**
- **True Positive = 1**
- **False Positive = 0**
- **False Negative = 0**

Interpretation

Accuracy = 100%

The model correctly classified both test employees.

Precision = 100%

Every employee predicted as likely to leave actually belonged to the leave class in the test set.

Recall = 100%

The model correctly identified the employee who was likely to leave.

F1-score = 100%

The model achieved a perfect balance between precision and recall on this test set.

Important: Since there are only **8 employees**, the test set contains just **2 employees**. Therefore, these 100% scores should not be considered proof that the model will achieve 100% accuracy on real HR data.

h) How HR Can Use the Prediction to Reduce Employee Attrition

The HR department can use the Logistic Regression model to identify employees who have a **higher probability of leaving**.

For example, the new employee has:

- Experience = 5 years
- Satisfaction Score = 4
- Overtime = 9 hours
- Salary Increment = 6%

The model predicts:

Employee may LEAVE

HR can take preventive actions such as:

1. **Improve job satisfaction** by discussing employee concerns.
2. **Reduce excessive overtime** and improve work-life balance.
3. **Review salary increments** and compensation.
4. **Provide career growth opportunities** and training.
5. **Conduct employee feedback sessions**.
6. **Offer recognition and rewards** for good performance.
7. **Provide mentoring and support** where needed.

Conclusion

Logistic Regression is suitable because employee attrition is a **binary classification problem**:

- **0 → Stay**
- **1 → Leave**

The model can help HR identify employees who may be at higher risk of leaving and take **early retention measures**. However, predictions should be used as a decision-support tool rather than as the sole basis for HR decisions, especially because the given dataset is very small.